Assignment 1 - Git & GitHub Fundamentals

Report

- Full Name: Fatima Ezzahra
- Student ID: 163591
- Course Code & Section: CSC 3324, section 2
- GitHub Username: fatimaezzahra456
- Repository: https://github.com/fatimaezzahra456/git-session1-fatimaezzahra456

1. Repository Setup

I copied the STARTER folder and renamed the copy git-session1-fatimaezzahra456, then initialized it with git init. The default branch is main. I created a .gitignore that ignores local.env (the environment file) and all .log files, with an exception line, !important.log, so that important.log stays visible to Git. The exception comes after *.log because Git reads the rules from top to bottom. I checked the result with git status, git check-ignore -v and git ls-files: local.env, app.log and debug.log are ignored, and important.log is tracked.

2. README & Commit History

My README.md contains my name, my GitHub username, a short description, a list of technologies, one code block (a git clone command), one link (my GitHub profile) and the provided banner image. I organized my work in small commits, each holding one idea: the .gitignore, the starter files, the README, a style change, a notes change, the About feature, the merge into main, and a README edit made on GitHub. Each commit message says what the commit does.

3. Selective Staging

I changed two tracked files at the same time, style.css and notes.txt. git diff showed both changes. I ran git add style.css only, and git status then showed style.css as staged and notes.txt as modified but not staged. git diff --staged showed only the style.css change. I committed style.css first, and notes.txt stayed modified until I committed it in a second commit. This works because the staging area lets me choose what goes into each commit.

4. Branch & Merge

I created a branch called about-feature with git switch -c about-feature and added an About section to index.html. I committed it on that branch, which left main unchanged. I then switched back to main and ran git merge --no-ff about-feature. I used --no-ff so that Git creates a merge commit, which keeps the branch visible in the history. Before the merge, searching index.html on main for the word about found nothing, and after the merge it found the new section.

5. Remote Changes

A commit saves a snapshot in my local repository only, while a push uploads my local commits to GitHub. After I edited README.md directly on GitHub, I ran git fetch, which downloads the new commit and updates origin/main without changing my files. git status then said my main was behind origin/main by 1 commit, and my README still showed the old sentence. git pull does a fetch followed by a merge, so it also updates my branch and my files. I ran git pull origin main, and the README then showed the new sentence. I also showed an unstaged change with git diff and a staged change with git diff --staged.

6. Fair Use of AI

I used Claude, an AI assistant, to guide me through this assignment step by step because Git and GitHub were new to me. I asked it to explain concepts in simple words (the staging area, branches, fetch versus pull), to explain what each command does and why, and to help me read my terminal output and fix mistakes, for example a .gitignore command that did not finish and README lines that were wrongly indented. I ran every command myself, wrote or changed the file contents and the commit messages myself, and did not accept anything I did not understand. I verified each step with git status, git diff, git log and git ls-files, and by checking the result on GitHub. I understand every command used in my submission.

7. Sources & Reflection

Sources: Session 1 slides, the assignment instructions, and the Atlassian gitignore tutorial linked in the slides.

Reflection: The hardest concept for me was the difference between my local repository and GitHub, especially fetch and pull. It became clear after I edited the README on GitHub, ran git fetch, and saw that my branch was behind origin/main while my files had not changed yet. If Git behaves unexpectedly, I first run git status to check my branch and whether I am ahead or behind, then git log --oneline --graph --all to see where my branches are.